SIES Graduate School of Technology
Sri Chandrasekarendra Saraswati Vidyapuram
Sector V, Nerul, Navi Mumbai - 400706

Name: Vansh Jain
Roll No: 124A8036
Class/Div: SE G

Experiment 7
Deadlock Avoidance Using Banker's Algorithm

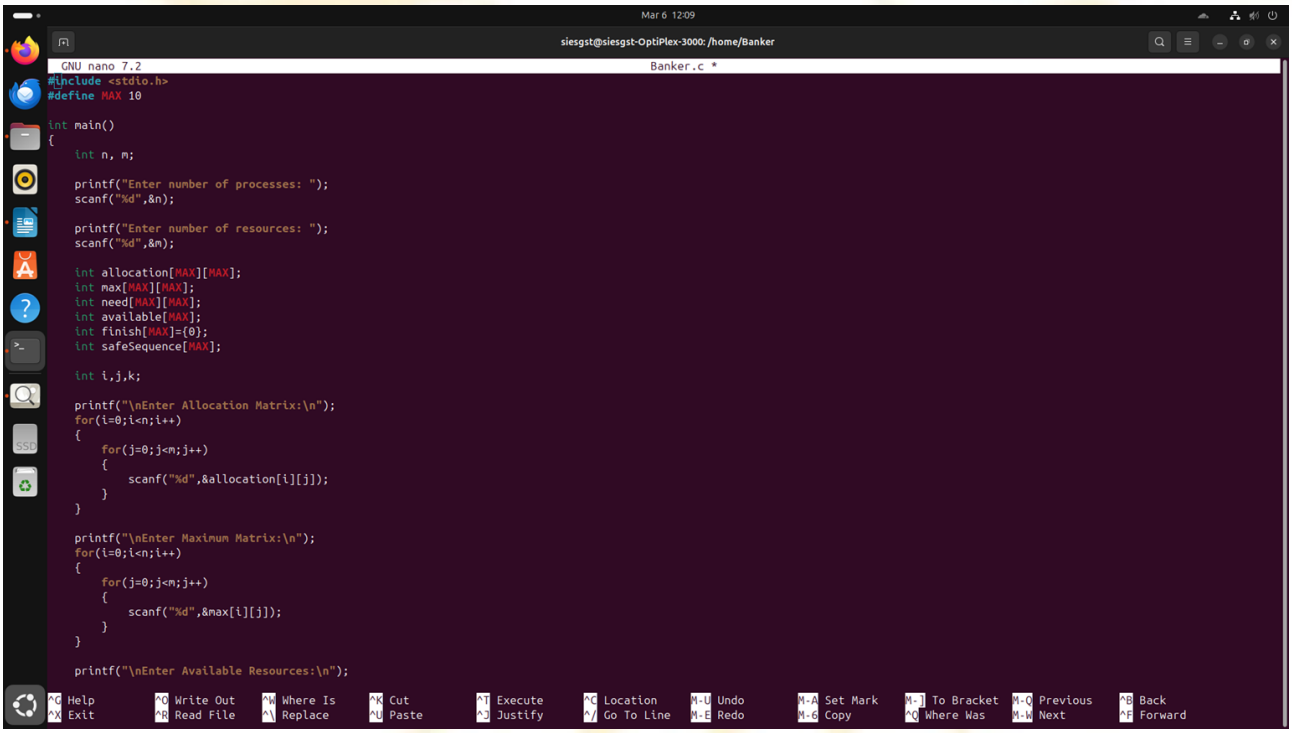
Aim:
To implement and demonstrate **deadlock avoidance using Banker's Algorithm** and determine whether the system is in a **safe state** by identifying a safe sequence of processes.

Theory:
In a multiprogramming system, several processes compete for limited resources such as CPU time, memory, and I/O devices. If resources are allocated improperly, the system may enter a **deadlock state** where processes wait indefinitely for resources held by each other.
Deadlock avoidance techniques ensure that the system **never enters an unsafe state**.
One of the most widely used deadlock avoidance techniques is **Banker's Algorithm**.
The algorithm works similarly to a bank that allocates loans to customers.
Before granting a resource request, the operating system checks whether allocating that resource will leave the system in a **safe state**.
If the system remains safe, the resource request is granted. Otherwise, the request is denied.
The algorithm uses the following data structures:
Allocation Matrix – shows currently allocated resources.
Maximum Matrix – maximum resources required by each process.
Need Matrix – remaining resources required by each process.
Available Vector – resources currently available in the system.
Need is calculated as:
Need = Maximum − Allocation
The system is safe if there exists a **safe sequence** where each process can complete using currently available resources.

Algorithm:
Step 1
Input the number of processes and resource types.
Step 2
Input the Allocation matrix.
Step 3
Input the Maximum matrix.
Step 4
Input the Available resources.

Step 5
Compute the Need matrix using:
Need[i][j] = Max[i][j] − Allocation[i][j]
Step 6
Initialize a Finish array for all processes as false.
Step 7
Find a process such that:
Need[i] ≤ Available
Step 8
If found, assume the process finishes and releases resources.
Available = Available + Allocation[i]
Mark Finish[i] = true.
Step 9
Repeat until all processes finish or no such process exists.
Step 10
If all processes finish → System is safe.
Otherwise → System is unsafe (deadlock possible).

Program:
Banker.



```
GNU nano 7.2                                   Banker.c *
#include <stdio.h>
#define MAX 10

int main()
{
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter number of resources: ");
    scanf("%d",&m);

    int allocation[MAX][MAX];
    int max[MAX][MAX];
    int need[MAX][MAX];
    int available[MAX];
    int finish[MAX]={0};
    int safeSequence[MAX];

    int i,j,k;

    printf("\nEnter Allocation Matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            scanf("%d",&allocation[i][j]);
        }
    }

    printf("\nEnter Maximum Matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            scanf("%d",&max[i][j]);
        }
    }

    printf("\nEnter Available Resources:\n");
^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket  M-Q Previous   ^B Back
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo      M-6 Copy       ^Q Where Was    M-W Next       ^F Forward
```

```
    for(i=0;i<m;i++)
    {
        scanf("%d",&available[i]);
    }

    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    int count=0;

    while(count<n)
    {
        int found=0;

        for(i=0;i<n;i++)
        {
            if(finish[i]==0)
            {
                int flag=1;

                for(j=0;j<m;j++)
                {
                    if(need[i][j] > available[j])
                    {
                        flag=0;
                        break;
                    }
                }

                if(flag==1)
```

```
                {
                    flag=0;
                    break;
                }
            }

            if(flag==1)
            {
                for(k=0;k<m;k++)
                {
                    available[k]+=allocation[i][k];
                }

                safeSequence[count]=i;
                finish[i]=1;
                count++;
                found=1;
            }
        }
    }

    if(found==0)
    {
        printf("\nSystem is NOT in safe state\n");
        return 0;
    }
}

printf("\nSystem is in SAFE state\n");
printf("Safe Sequence: ");

for(i=0;i<n;i++)
{
    printf("P%d ",safeSequence[i]);
}

printf("\n");

return 0;
}
```

Output:

```
siesgst@siesgst-OptiPlex-3000:~$ ./banker
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in SAFE state
Safe sequence: P1 P3 P4 P0 P2 siesgst@siesgst-OptiPlex-3000:~$ nano dining.c
```

Viva Questions

1. What is Deadlock?

Deadlock is a situation in an operating system where two or more processes are waiting indefinitely for resources held by each other, so none of the processes can proceed. The system becomes stuck because every process in the set is waiting for another process to release a resource.

2. What are the Four Necessary Conditions of Deadlock?

Deadlock occurs only when all four conditions exist simultaneously:

1. **Mutual Exclusion** – At least one resource cannot be shared and must be used by only one process at a time.
2. **Hold and Wait** – A process holding at least one resource is waiting to acquire additional resources held by other processes.
3. **No Pre-emption** – Resources cannot be forcibly taken away from a process; they must be released voluntarily.
4. **Circular Wait** – A circular chain of processes exists where each process waits for a resource held by the next process.

3. What is a Safe State?

A safe state is a condition where the system can allocate resources to processes in some order and avoid deadlock. In a safe state, there exists a safe sequence of processes such that each process can complete its execution without causing deadlock.

4. Difference Between Deadlock Prevention and Deadlock Avoidance?

Deadlock Prevention	Deadlock Avoidance
Prevents deadlock by eliminating one of the four necessary conditions.	**Avoids deadlock by carefully allocating resources based on the current system state.**
More restrictive and may reduce system performance.	**Uses algorithms (like Banker's Algorithm) to check safety before allocation.**
Example: forcing processes to request all resources at once.	**Example: granting a request only if the system remains in a safe state.**

5. Why is Banker's Algorithm called Banker's Algorithm?

It is called Banker's Algorithm because it works like a bank lending money. The bank only gives loans if it is sure that all customers can still repay their loans without causing bankruptcy. Similarly, the algorithm allocates resources only if the system remains in a safe state.

6. What is the Need Matrix?

The Need Matrix represents the remaining resources required by each process to complete its execution.

Formula:

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

Where:

- Max = Maximum resources a process may need
- Allocation = Resources currently allocated
- Need = Remaining resources required

7. **What happens if the system enters an Unsafe State?**
   If the system enters an unsafe state, it does not immediately mean deadlock has occurred, but there is a possibility of deadlock in the future. If processes request resources in certain sequences, the system may not be able to satisfy them, leading to deadlock.

Dining Philosopher's Problem

Program:

Philosopher.c

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];

void* philosopher(void* num)
{
    int id = *(int*)num;

    while(1)
    {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id+1)%N]);

        printf("Philosopher %d is eating\n", id);
        sleep(1);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id+1)%N]);
    }
}

int main()
{
    pthread_t thread_id[N];
    int phil[N];
    int i;

    for(i=0;i<N;i++)
    {
        sem_init(&chopstick[i],0,1);
    }

    for(i=0;i<N;i++)
```

```c
    for(i=0;i<N;i++)
    {
        sem_init(&chopstick[i],0,1);
    }

    for(i=0;i<N;i++)
    {
        phil[i]=i;
        pthread_create(&thread_id[i],NULL,philosopher,&phil[i]);
    }

    for(i=0;i<N;i++)
    {
        pthread_join(thread_id[i],NULL);
    }

    return 0;
}
```

Output:

```
siesgst@siesgst-OptiPlex-3000:/home/Banker$ ./philosophers
Philosopher 0 is thinking
Philosopher 2 is thinking
Philosopher 1 is thinking
Philosopher 4 is thinking
Philosopher 3 is thinking
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 1 is thinking
Philosopher 3 is thinking
Philosopher 2 is eating
Philosopher 0 is eating
Philosopher 2 is thinking
Philosopher 0 is thinking
Philosopher 1 is eating
Philosopher 4 is eating
Philosopher 1 is thinking
Philosopher 0 is eating
Philosopher 4 is thinking
Philosopher 3 is eating
Philosopher 0 is thinking
Philosopher 3 is thinking
Philosopher 2 is eating
Philosopher 4 is eating
Philosopher 2 is thinking
Philosopher 4 is thinking
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 1 is thinking
Philosopher 3 is thinking
Philosopher 2 is eating
Philosopher 0 is eating
Philosopher 2 is thinking
Philosopher 0 is thinking
Philosopher 1 is eating
Philosopher 4 is eating
Philosopher 1 is thinking
Philosopher 4 is thinking
Philosopher 0 is eating
Philosopher 3 is eating
Philosopher 0 is thinking
Philosopher 3 is thinking
```